QCon
全球软件开发大会

InfoQ 技术大会

当代码不再稀缺

大型工程 AI 狂欢下的确定性底座

张忻正

bilibili 技术专家

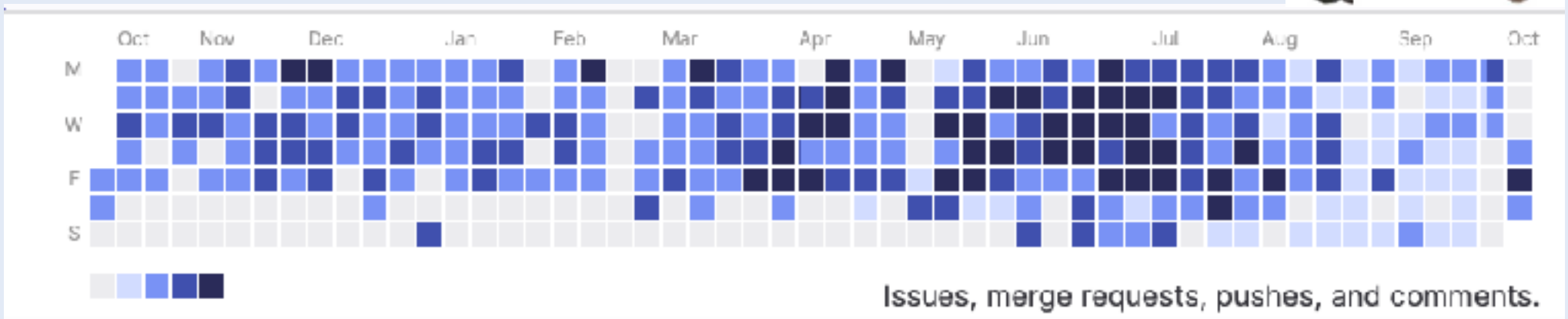
QCon 全球软件开发大会 2026 · 上海 | AI 驱动的软件工程



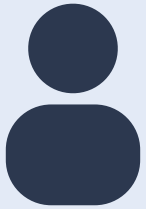
关于我

- **现在负责** bilibili 大前端架构，和主站业务架构
- **一直在做** 基于Monorepo的大型复杂软件工程
- **深耕一线** 所有人，都过来！





团队突然多出 100 倍的人，你放心吗？

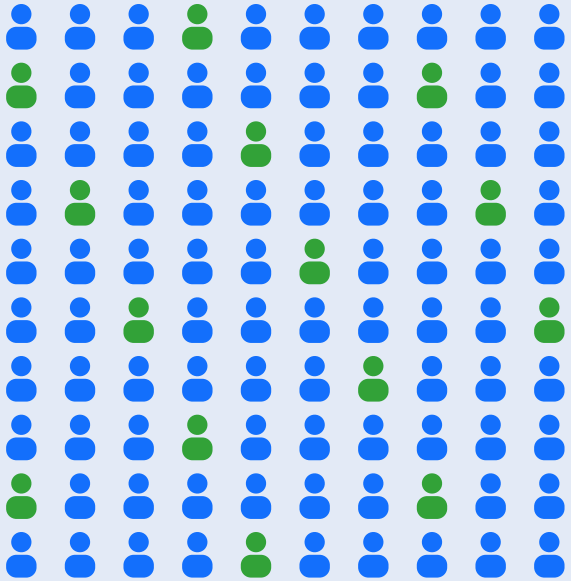


以前

一个人写

一个人 review





现在：每个人背后还有好几个 Agent

review 还看得过来吗？

CI 还排得过来吗？

文档还跟得上吗？

这不是假设：400 人在一个仓里，九成以上在用 Agent



大型！复杂！软件工程！

数字	意味着什么
约 400 人，九成以上在用Code Agent	CodeAgent早就不是试点了
App上 一个构建目标(Binary) 14.3 万 action	改一行影响多大，人算不出
294 万条引用，78.6 万个声明	谁能调谁，人记不住，文档也记不住
关键路径 64 层串行编译	加机器也得一层层等（scalable是种理想的奢望）

构建数字以 iOS 旗舰版为例，后面会展开



B站全栈化后的组织架构

业务结果团队	国内业务		出海业务		
领域交付 与核心能力	商业与交易交付	用户与增长交付	社区与互动交付	稿件服务交付	视频与媒体网络
AI-Native 研发基础设施	工程交付平台	知识工程平台	质量工程	工作流编排平台	客服系统

① 业务定义结果　② 领域端到端交付　③ AI-Native 基座

SYNC
同期率
度
验
定
约
MENU

QCon
全球软件开发大会

InfoQ 技术大会

目录

01 AI Coding 在大型工程中为什么容易失速

02 AI 时代工程能力的四个底座

03 大型团队的真实实践与取舍

SYNC
--.-
同期率
度
验
定
约
MENU

01

AI Coding 在大型工程中为什么容易失速

放大器 · 反馈循环 · 验证债 · Harness 腐化



AI 是放大器，不负责补短板

工程扎实的地方

规则机器能查

结果能复现

× AI

交付真的变快

快 10 倍、100 倍都有可能

规则只写在文档里的地方

靠人记、靠人看

× AI

问题扩散得一样快

回头再收拾，代价 10 倍不止

放大器不会帮你补规则，只会让缺规则的后果来得更快



好例子：Chrome 的分层靠机器守



DEPS 文件

每个目录写明：这里能 include 谁

presubmit 里的 checkdeps

违规的 include，提交前就被拦下

Mojo 接口文件

跨进程先写 .mojom，再生成各端代码

分层规则写在构建里：人换了几茬，规则还在

SYNC
同期率
度
验
定
约
MENU

再看我们：同一个 B 站，做了N遍

同一个 B 站	老 Android	老 iOS	Web
网络和数据	这项赢 接口 + 数据类 字段写错，编译就过不了	字符串取字段 写错了，要到线上才知道	类型手写在 .d.ts 里 不从契约生成
异步	这项赢 协程为主 2875 处	回调为主 5487 处	async/await 约 2487 处 .then 还有 470 处
直播间	逻辑拆了，物理上 一个 2251 文件的大模块	这项赢 拆成 957 个模块 零循环依赖，编译期掐住	逻辑拆了，物理上 一个约 40 万行的大模块

AI 进了哪个仓，就学会哪一套



「仓库的文件，

就是 AI 的"老师"。」

人类的本质就是个复读机



给 AI 配的规则和工具，会烂，且烂得很快

- **文档会过时** 代码改了文档没改，AI 拿着旧信息越写越自信

  实测：《Detecting outdated code element references in software repository documentation》研究发现，在所分析的 800 个项目的历史中，82.3% 曾出现文档与代码不一致的情况

- **老规则会拖后腿** 模型变强了，当初的限制可能变成绊脚石

  参考：OpenAI《Rethinking skills and prompts for GPT-6 Astra》 https://developers.openai.com/blog/rethinking-skills-and-prompts-for-gpt-6-astra

- **补丁越贴越多** 最后没人说得清，到底是哪条规则在起作用

  论文《Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?》：上下文文件并没有普遍提高任务成功率，推理成本平均还增加 20% 以上



反馈慢：64 层编译，一层一层等

一次干净构建的墙上时间 · 10 个样本

关键路径 95.5%：64 层串行编译

能做的	拆模块，把长链打断
没用的	加机器、加并发，链还是一层一层等
反直觉的	单测有收益的优化，落在关键路径上就吃不到

瓶颈不在机器，在那条串起来的长链

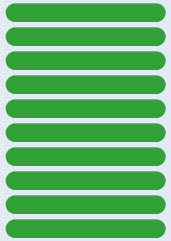
SYNC
--.-
同期率
度
验
定
约
MENU

开 10 个 Agent，不等于快 10 倍

1 个 Agent

改 → 等构建、等测试、等真机 → 看结果

10 个 Agent



同一条队：排队等反馈

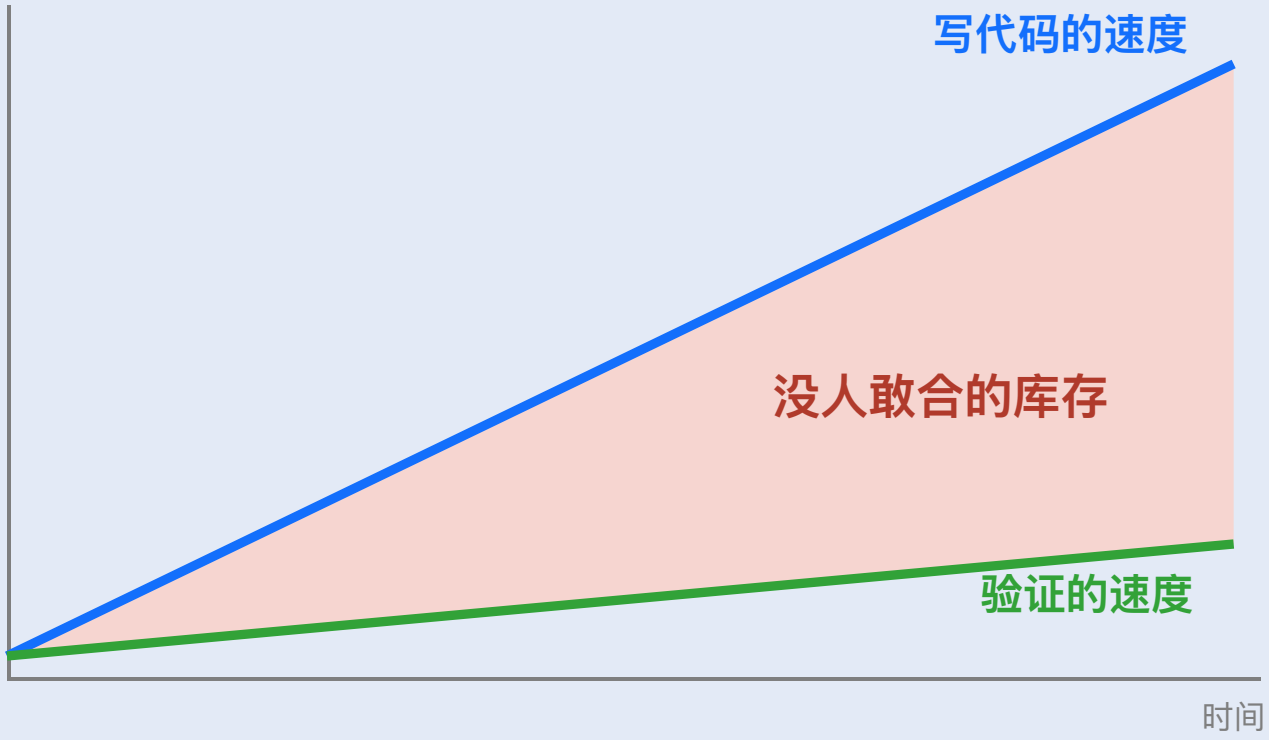
攒出 10 份没验证的改动，还可能互相改到同一处

→ 看结果

要压的是「改完、跑一遍、看结果」这一轮，压到分钟级

SYNC
同期率
度
验
定
约
MENU

写得越快，没人敢合的代码越多



以前

写得慢，review 还跟得上

现在

出码翻了几倍

review、真机回归一点没变快

代码写完不算完

证据齐了才算完

SYNC
同期率
度
验
定
约
MENU

先补底数

可信的反馈

影响面算得出 · 构建能复现 · 验证能自动跑

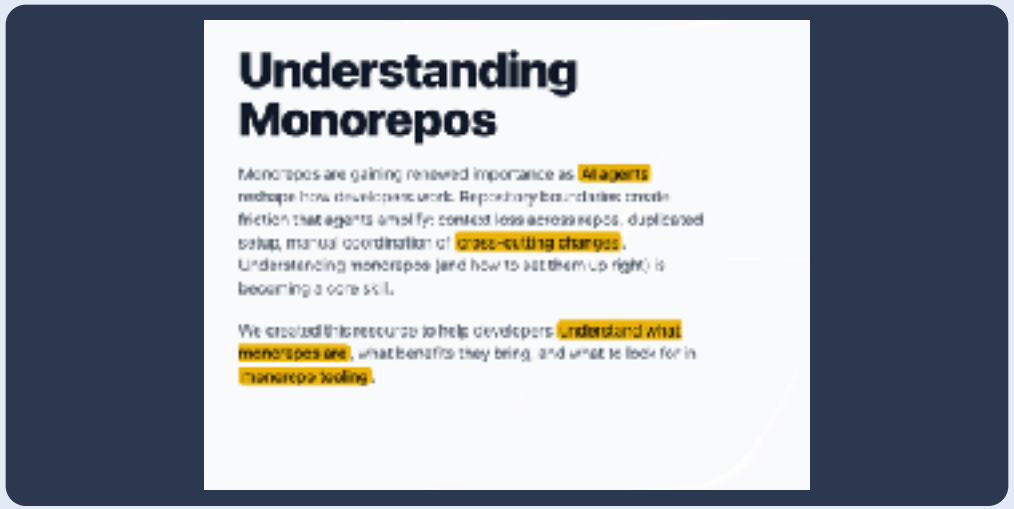
AI

写 · 查 · 改

=

能放心的交付速度

每一轮都更快



旁证：monorepo.tools

B战的底数?

移动端、Web、后端各进一个大仓

统一的构建和测试入口

代码关系、环境、验证都在一处，

AI 才拿得到完整的上下文

SYNC
同期率
度
验
定
约
MENU

02

AI 时代工程能力的四个底座

约束 · 定位 · 验证 · 度量　把工程规则拆成 Agent 可执行的交付闭环



四个底座：从工程规则到交付决策

约束

「什么能写」

规则写进构建

机器来查

定位

「该看哪里」

让构建图划范围

别硬塞上下文

验证

「凭什么说对」

证明用户

真的看见了

度量

「敢不敢合」

只数直接能合

的 MR

四件事合起来，只回答一句：这个 MR 敢不敢直接合

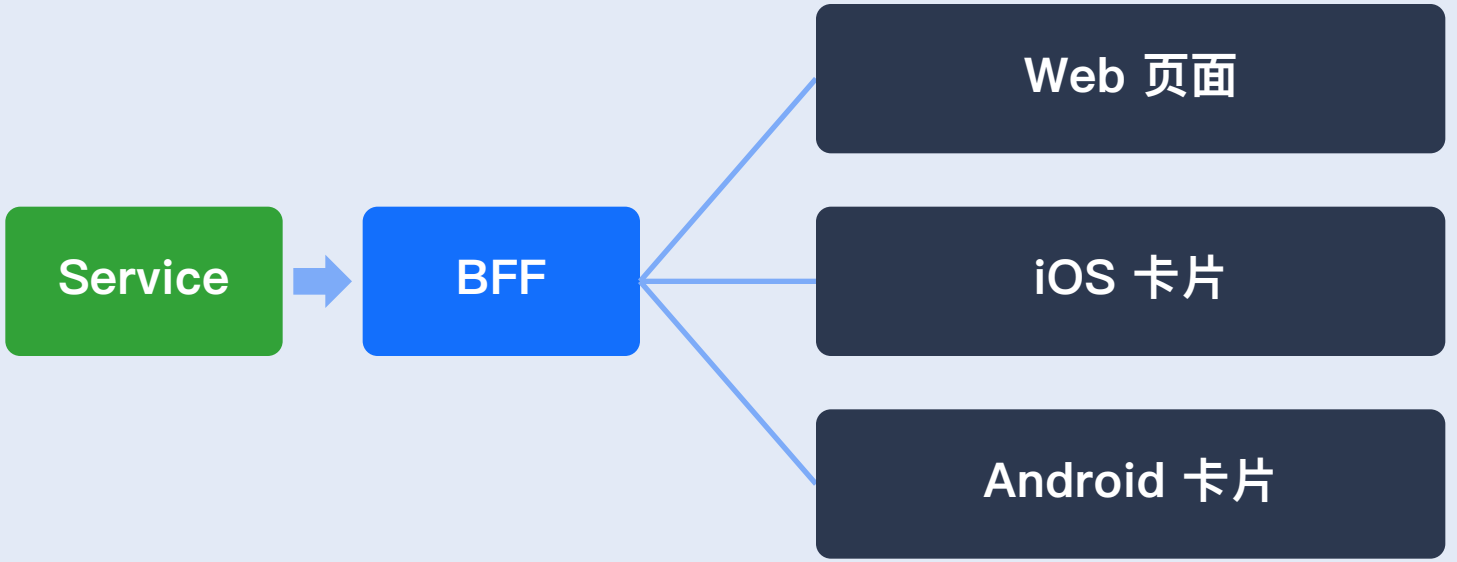
顺序不能乱：前一步没做，后一步只能靠猜



例子：首页卡片加一个「新」角标



首页卡片，加一个「新」角标



听着就是加个字段，要穿过 5 个地方

后面每一步，都拿它当例子

SYNC
同期率
度
验
定
约
MENU

约束 ①：没有契约，每一个人各写一份

后端 BFF（Go）

```
type Card struct {
  Title string `json:"title"`
  Badge string `json:"badge"`
}
```

AI 先加了字段，本地一跑就通

iOS（Swift）

```
struct Card: Codable {
  let title: String
  let badgeText: String?  // 名字写错
}
```

对不上，永远是空，角标不显示，也不报错

Android（Kotlin）

```
data class Card(
  val title: String,
  val badge: Int  // 类型写错
)
```

解析时报错，这张卡直接出问题

Web（TypeScript）

```
interface Card {
  title: string
  // 漏写了 badge
}
```

页面上没有角标，类型检查也不报错

契约里没有，各端就各写各的：漏了、写错了，都要跑起来才知道

SYNC
23.7
同期率
度
验
定
约
MENU

约束 ②：先改契约，代码生成出来

契约

proto 里加一个 badge 字段

这是唯一的源头

构建时生成

后端、BFF、各端的代码

都从契约生成，不提交

业务代码

只用生成出来的类型

不许手写一份

CI 检查：兼容性 diff · 生成代码不许手改 · 契约测试

AI 要改字段，先改契约；两端要动哪里，生成和检查会告诉它

SYNC
23.7
同期率
度
验
定
约
MENU

同样的事，语言层也能做

老写法：C++ 回调

```
class SomeObject {
  // 异步结果：只能靠回调传出去
  void doSth(function<void(int)> cb) {
    cb(42);
  }
  // 持续状态：自己存，手动注册、反注册
  void registerCb(function<void(int)> cb);
  void unregisterCb();
};
```

新写法：Kotlin 协程 + Flow

```
class SomeObject {
  // 异步结果：挂起函数，跟同步写法一样
  suspend fun doSth(): Int = 42

  // 持续状态：一条 Flow，跟着 scope 走
  val state: Flow<Int>
  // 取消、错误往上抛，交给 CoroutineScope
}
```

流程变直了

异步写起来跟同步一样

生命周期有人管

任务能取消，错误能往上抛

状态说得清

持续状态和一次性事件分开

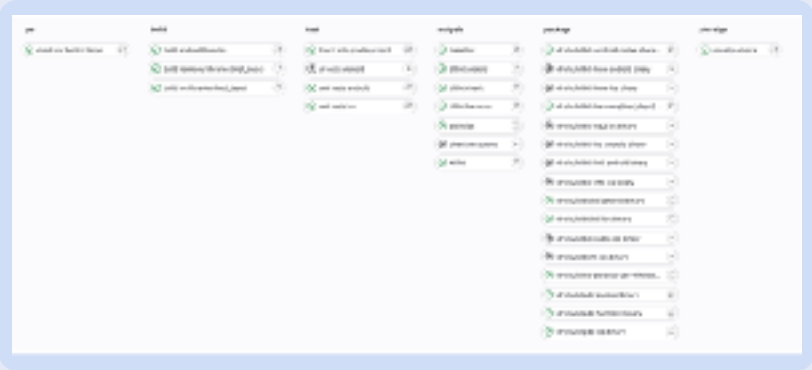
不用兜底了

Caller的职责交给Caller，而不是Callee承担

原来靠自觉遵守的约定，现在由语言来守

SYNC
23.7
同期率
度
验
定
约
MENU

约束 ③：本地就跑和 CI 一样的检查



我们现在的 pipeline

pre	分支有没有落后主干太多
build	所有配置下都能编过
test	单测、UI 测试还过不过
analysis	依赖边界和老毛病有没有再犯
package	旗舰版、国际版能不能打出包
pin-align	跨仓契约的版本对不对得上

Agent 在本地就能跑一样的检查，红灯来得越早越便宜

SYNC
23.7
同期率
度
验
定
约
MENU

定位 ①：改了契约，会影响谁?

靠猜

全仓搜 badge，冒出一堆同名字段。

问 RAG，索引可能还是上周的。

研发凭经验报：双端都看看吧。

直接问构建图

谁依赖了这份契约?

最后打进了哪些包?

要跑哪些测试?

直接定位有多重要，可以参考：知乎《为什么现在 Agent 重新用回 Grep，而不是先做 RAG?》
https://www.zhihu.com/question/2056838324170514762/answer/2058548538620093658



定位 ②：构建图直接给出答案

问构建图　`bazel query "rdeps(//..., :card_proto)"`

受影响的模块

iOS 首页卡片

Android 首页卡片

会打进哪些包

旗舰版

国际版

要跑的测试

卡片单测

卡片 UI 测试

交给 QA：要测首页卡片，双端，两个包

不用测：播放页、直播间，没有依赖关系

Web 那边，Turborepo 也能列出依赖它的包，思路一样

SYNC
46.2
同期率
度
验
定
约

再看一个日常例子：构建挂了，先看 action

只给 Agent 看源码：它很容易修错

错误：找不到生成的 serializer
→ 看当前文件和相似写法
→ 改注解、补 import、甚至手写兜底
→ 根因不在源码，越修越偏

先重放失败的 action

```
bazel aquery 'deps(//binary/...:旗舰版目标)' --config=android
```
→ 看到这次编译真实的命令行
→ 里面没有 serialization 编译插件

正确的修法：改构建配置，不动业务代码

给这个 target 补上编译插件
→ 重跑同一个 action，再跑受影响的包

搜代码给的是文本，aquery 给的是这次失败时真正执行了什么

SYNC
46.2
同期率
度
验
定
约

「我们正陷入一场，

与人（AI）的信任拉锯战。」

编过了、单测绿了，只能证明一小块

验证 ①：在 Web 上证明「用户看见了」

造数据

fixture：一张带角标的卡片

下游依赖用 mock

账号、路由固定

走真实接口

APITest 调真实 HTTP 入口

断言返回里有 badge

开真浏览器

按真实路由打开页面

CDP 断言页面上有角标

一条用例，四份证据：造的数据 · 接口返回 · 页面 DOM · 网络和 Console

Agent 改完自己跑一遍，不用再叫人点开页面看



验证 ②：错了，Agent 知道改哪一层

Agent 看到的结果	它去改哪里
接口返回里没有 badge	改后端或 BFF
接口有，页面上没有	改 Web 的渲染
接口有，页面也有	通过，进入下一步

前后端放在同一个环境里跑，每一跳的日志串在同一条 trace 上，断在哪一眼就看得到

SYNC
68.5
同期率
度
验
定
约
MENU

验证要便宜：统一构建后，验证次数翻倍

同一批研发 · 同一套构建集群 · 没加机器

iOS

构建耗时（分钟）

P50	13 →	7	−46%
P80	24 →	12	−50%
P90	34 →	18	−47%

Android

构建耗时（分钟）

P50	22 →	11	−50%
P80	25 →	17	−32%
P90	34 →	22	−35%

iOS 每天的构建次数

约 180 → 约 360

验证次数翻倍

33 份反馈里 24 份
说 AI 用起来顺了

验证又快又可信，研发和 Agent 才肯多跑

SYNC 68.5 同期率 度 验 定 约 MENU

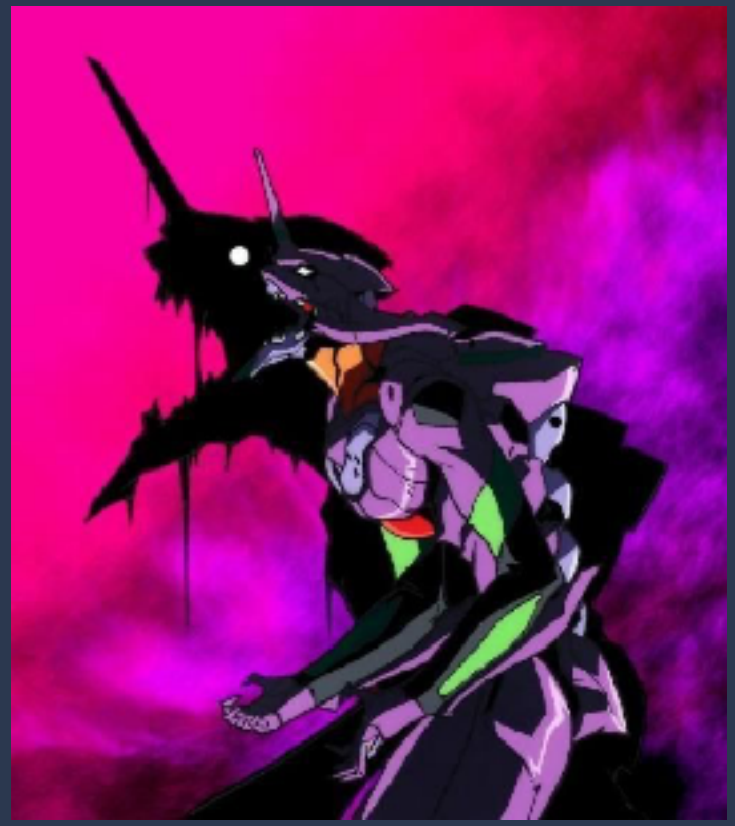
度量：这个角标 MR，能直接合吗？

后端 + BFF	Web 页面	iOS / Android
✓ 契约检查过了	✓ 接口返回有 badge	✓ 编译过了
✓ 接口测试过了	✓ 页面上有角标	✓ 单测绿了
✓ 开关随时能关	✓ 网络和 Console 干净	✗ 用户看没看见？
直接合	证据齐，机器能判	CODEOWNER 不签



代码都写对了，差的是最后那一条证据

「没用！ ...」



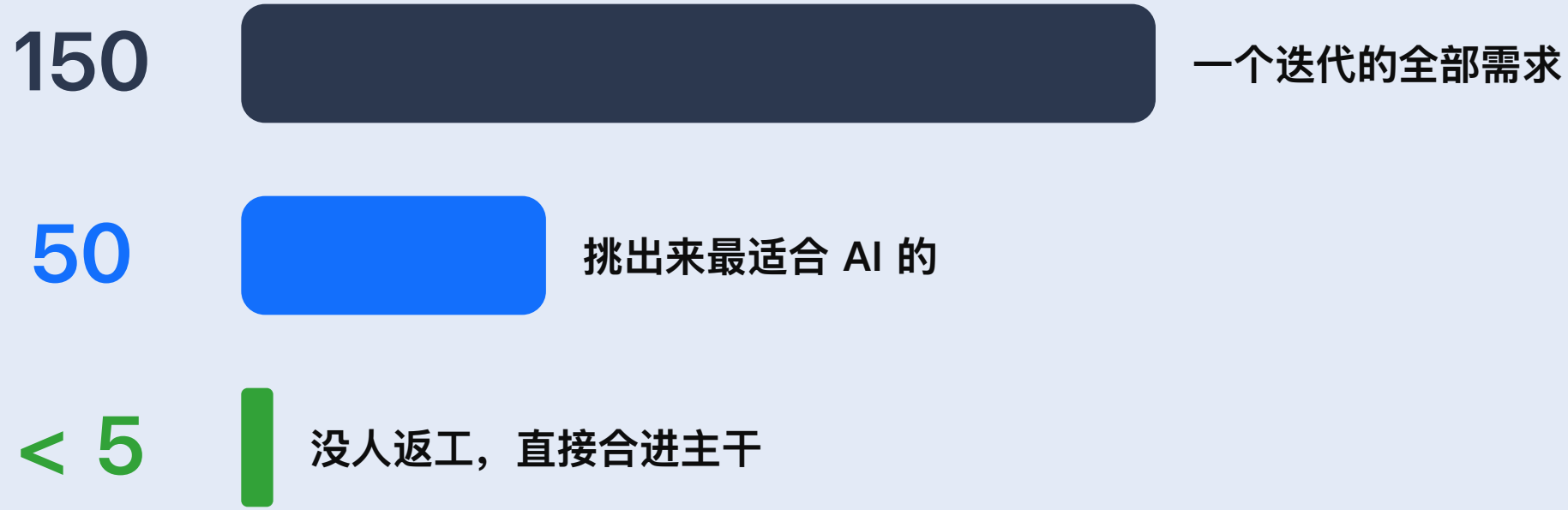
03

做对、止损与下一步

统一工程 · 统一验证入口 · 用证据决定 Agent 能走多远



Human out of the loop?



补证据、改代码、重跑验证，有一样就不算

能直接合的，都是错了能马上撤回的

改动类型	出错了怎么撤回	直接合并的结果
后端实验开关、和界面无关的逻辑	拨开关就能关，不用发版	能直接合的，全在这一类
只要涉及客户端的改动	只能等下个版本，或者热修	一个都没能直接合

其中有些客户端改动结果完全正确，CODEOWNER 也没有签字

SYNC
400%
暴走
度
验
定
约
MENU

客户端卡在哪：拿不出「用户看见了」

Web：页面跑起来就是 DOM

Agent 用 DevTools 读页面、点按钮、看网络

本地、CI、Agent 用同一个接口

能断言：用户看见了角标

客户端：没有统一的运行面

iOS、Android 的 UI 树和调试协议各不一样

真机、系统版本、账号、网络，变数太多

只能说：编译过了，单测绿了

CODEOWNER 不签，是因为手上真没东西可看



迟到的工程（工业）化

哪个仓	统一用的工具	Agent 因此能做到
移动端	Bazel，加远程缓存和远程执行	改完几分钟就能拿到构建和测试结果
Web	Turborepo	任务有缓存，还能用浏览器的调试接口看页面
后端	Go 统一工程	依赖可以造数据，接口可以真实调用来测试

回报看前面那组数：同样的人和机器，终端 每天的构建次数翻了一倍，耗时少了一半



六个 Agent，只留下一个

留 Code Agent

日常写代码、改代码

唯一全员铺开的

砍 需求澄清 Agent

需求没固定到能套模板

绕过人做计划没意义

砍 Custom-code Agent

成熟的通用 Agent 够用

不重复造

并 Debug Agent

不单独成一个 Agent

变成 Code Agent 的工具

在建 Verify Agent

给客户端补上

「用户看见了」的证据

规划 Deploy Agent

把证据接到线上

管灰度和回滚

还砍了很多好看的指标- -

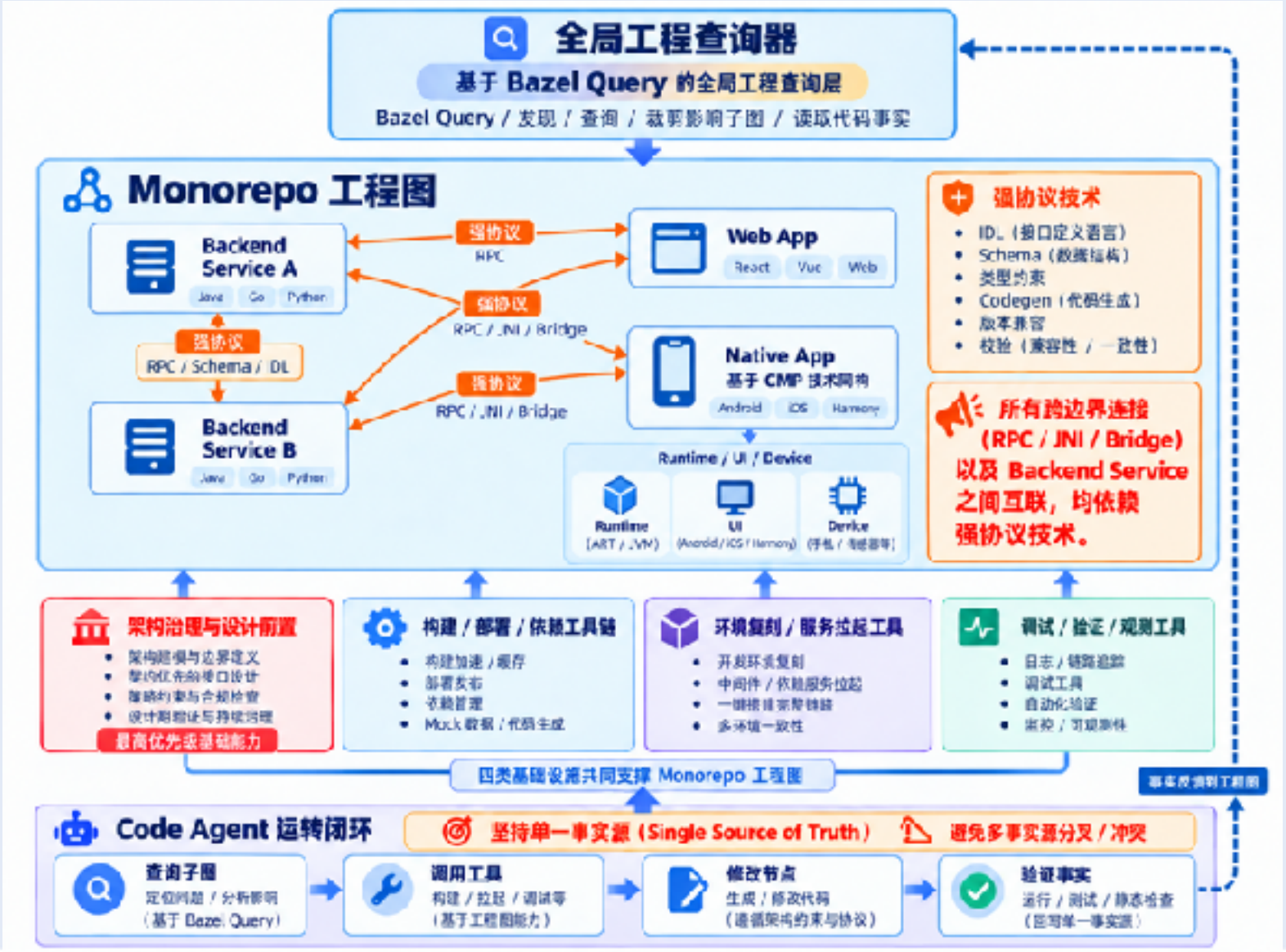


SUMMARY

工程化

自动化

智能化



SUMMARY

交付能力 = 工程化

智能化

自动化

（确定性工程）



底数大了，指数的放大才够；底数太小，指数再大也放不大；
底数不及格，指数？...



「代码不再稀缺之后，

最稀缺的是让人放心的交付。」

这个人、这个团队、这套规则、这个 Agent，靠不靠谱

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师



Thanks

本演示文稿由 Claude Code 与 Codex 协作制作